



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Dipartimento di Informatica Scienza e Ingegneria

Corso di Laurea in Informatica

Traduzione da linguaggio naturale a query per Blazegraph

Relatore:
Chiar.mo Prof.
Fabio Vitali

Presentata da:
Alberto Zuccari

I Sessione 2026
Anno Accademico 2025/2026

(DA FARE ALLA FINE)

5 parole chiave per caratterizzare il contenuto della dissertazione:

parola 5

parola 4

parola 3

parola 2

Parola 1

*Non tocca a noi dominare tutte le maree del mondo
il nostro compito è di fare il possibile
per la salvezza degli anni nei quali viviamo*

- J.R.R. Tolkien

Abstract

Abstract qui (da completare alla fine).

Indice

1	Introduzione	1
1.1	Il contesto e il problema	1
1.2	Struttura della dissertazione	1
2	Contesto scientifico e tecnologico	3
2.1	Introduzione al capitolo	3
2.2	Stato dell'arte: dall'interrogazione formale alla ricerca semantica . . .	3
2.2.1	Il contesto: Digital Humanities e la barriera della ricerca . . .	3
2.2.2	Il divario tra ricerca strutturata e ricerca esplorativa	4
2.2.3	La "scatola vuota" e le tre barriere di SPARQL	4
2.3	Fondamenti di architettura dell'informazione	5
2.3.1	L'illusione della ricerca algoritmica	5
2.3.2	Recall, Precision e il compromesso ineluttabile	5
2.3.3	Carico cognitivo: la Legge del minimo sforzo e la Legge di Hick .	6
2.3.4	Paradigma classico: navigazione e filtri rigidi	7
2.3.5	Il nuovo paradigma: ricerca semantica con LLM	7
2.4	Retrieval-Augmented Generation (RAG) come soluzione tecnologica .	8
2.4.1	Il paradigma RAG tradizionale	8
2.4.2	RAG applicato a Knowledge Graphs	8
2.4.3	Fine-tuning e LoRA per Domain Adaptation	9
2.5	Conclusioni del capitolo	9
3	Il prototipo	11
3.1	Introduzione al capitolo	11
3.2	Il dominio e gli utenti target	11
3.2.1	Il contesto operativo: DH.ARC e il Datavault	11
3.2.2	Profilo dell'utente finale: i ricercatori del centro	12
3.3	Architettura Funzionale e Interfaccia Utente	12
3.3.1	L'interfaccia: come si usa il Data Vault Explorer	12
3.3.2	Il flusso dell'interazione	13
3.3.3	I componenti del sistema e le loro responsabilità	14
3.4	Casi d'uso principali ed esempi	14
3.4.1	Caso d'uso 1: ricerca per caratteristiche di acquisizione e gestione anomalie	14
3.4.2	Caso d'uso 2: disambiguazione semantica dell'autore	15

3.4.3	Caso d'uso 3: ricerca multi-criterio con filtro temporale	16
3.5	Conclusioni del capitolo	16
4	Architettura e dettagli implementativi	17
4.1	Introduzione al capitolo	17
4.2	Stack tecnologico: Fedora e Blazegraph	17
4.2.1	I tre layer del sistema	18
4.2.2	Il flusso end-to-end di una richiesta	18
4.2.3	Scelte tecnologiche principali	19
4.3	Pipeline di fine-tuning dell'LLM	19
4.3.1	Dataset generativo e schema SPARQL	19
4.3.2	Configurazione dell'addestramento con Unsloth	22
4.4	Architettura del backend e integrazione frontend	25
4.4.1	Inference engine (FastAPI)	25
4.5	Formato dati: dataset_datavault.jsonl	30
4.5.1	Le 8 regole di generazione SPARQL	30
4.5.2	Gestione degli errori: OUT_OF_DOMAIN	30
4.6	Conclusioni del capitolo	31
5	Valutazione	33
5.1	Introduzione al capitolo	33
5.1.1	Metodologia generale	33
5.2	Efficienza: valutazione quantitativa	34
5.2.1	Struttura dei test	34
5.2.2	Distribuzione del test set per categoria	34
5.2.3	Tempo di generazione	35
5.2.4	Tasso di esecuzione corretta	35
5.2.5	Convergenza del training	36
5.3	Efficacia: valutazione qualitativa	37
5.3.1	Metriche di retrieval per categoria	37
5.3.2	Nota critica: interpretazione dei risultati perfetti	37
5.3.3	Miglioramento rispetto alle soluzioni precedenti	38
5.4	Discussione dei risultati e limiti della valutazione	39
5.5	Conclusioni del capitolo	39
6	Conclusioni e sviluppi futuri	41
6.1	Risultati raggiunti	41
6.2	Motivazioni di orgoglio e modestia	41
6.2.1	Punti di forza e orgoglio	41
6.2.2	Limiti del sistema e punti di modestia	41
6.3	Sviluppi futuri	41
6.3.1	Bug fix e ottimizzazioni a breve termine	41
6.3.2	Estensioni fondamentali e feature mancanti	41
6.3.3	Visioni di lungo termine (Desideri e sogni)	41
	Riferimenti bibliografici	43

Elenco delle figure

3.1	L'interfaccia principale del Data Vault Explorer con la LLM Search Bar.	13
3.2	Schema del flusso di interazione tra utente, LLM Query Builder e Datavault.	14
3.3	Caso d'uso 1: A sinistra la query SPARQL generata e sanitizzata, a destra l'estrazione delle risorse corrispondenti al dispositivo.	15
3.4	Caso d'uso 2: Disambiguazione semantica dell'autore e visualizzazione dei risultati.	15
3.5	Caso d'uso 3: Applicazione logica del range temporale ed estrazione delle immagini.	16
4.1	Architettura logica e flusso di comunicazione del sistema.	17
5.1	Distribuzione del test set di valutazione (n=500).	35
5.2	Distribuzione dei tempi di generazione su GPU T4.	35
5.3	Andamento della training loss e della validation loss durante il fine-tuning	36

Elenco delle tabelle

4.1	Le 13 categorie di query SPARQL generate da <code>genera_dataset.py</code> , più la categoria Out-of-Domain gestita a parte nel ciclo di generazione.	21
4.2	Iperparametri di fine-tuning LoRA (1/2): modello base e configurazione dell'adapter LoRA, configurato tramite Unsloth.	22
4.3	Iperparametri di fine-tuning LoRA (2/2): configurazione del training tramite <code>SFTTrainer</code> (libreria TRL).	23
4.4	Riepilogo dei principali casi di errore gestiti dal sistema e relative strategie di mitigazione adottate.	31
5.1	Distribuzione del test set per categoria euristica.	35

5.2	Statistiche del tempo di generazione per query (GPU T4).	35
5.3	Precision, recall e F1 medi per categoria, calcolati sull'insieme completo dei risultati (al netto di <code>ORDER BY / LIMIT</code>)	37

Elenco dei codici

4.1	Struttura del ciclo di generazione del dataset	20
4.2	Caricamento del modello base	23
4.3	Configurazione dell'adapter LoRA tramite Unsloth	24
4.4	Configurazione del training tramite SFTTrainer (libreria TRL)	24
4.5	Caricamento e fusione dell'adapter LoRA all'avvio del server FastAPI	25
4.6	Route di generazione della query SPARQL	26
4.7	Implementazione del sanitizzatore nel backend FastAPI.	28
4.8	System prompt utilizzato in training e inference (identico in entrambi i file)	30

Capitolo 1

Introduzione

1.1 Il contesto e il problema

1.2 Struttura della dissertazione

Capitolo 2: Contesto scientifico e tecnologico

Capitolo 3: Il prototipo

Capitolo 4: Architettura e implementazione

Capitolo 5: Valutazione

Capitolo 6: Conclusioni e sviluppi futuri

Capitolo 2

Contesto scientifico e tecnologico

2.1 Introduzione al capitolo

Il capitolo esplora il background scientifico e tecnologico alla base del lavoro proposto. Partiamo dal *problema fondamentale*: come fornire accesso content-aware a repository RDF di dimensioni massive per utenti privi di competenze tecniche formali?

La risposta richiede una comprensione su tre livelli di complessità:

1. **Lo stato dell'arte**: come gli approcci precedenti (form rigidi, endpoint SPARQL nudi) falliscono nel dominio umanistico;
2. **I fondamenti teorici**: i principi dell'architettura dell'informazione che guidano un'alternativa;
3. **Le tecnologie emergenti**: come Retrieval-Augmented Generation (RAG) e Large Language Models (LLM) abilitano una soluzione più flessibile.

Procederemo così: nel paragrafo 2.2 analizziamo il divario tra i bisogni esplorativi degli studiosi umanisti e la rigidità sintattica dei sistemi tradizionali. Successivamente, nel paragrafo 2.3, esaminiamo i principi fondativi dell'architettura dell'informazione che giustificano il ricorso agli LLM. Infine, nel paragrafo 2.4, descriviamo il paradigma RAG come soluzione tecnologica che bilancia flessibilità e affidabilità.

2.2 Stato dell'arte: dall'interrogazione formale alla ricerca semantica

2.2.1 Il contesto: Digital Humanities e la barriera della ricerca

Nel dominio delle Digital Humanities (DH) e della gestione del patrimonio culturale, la ricerca all'interno di repository RDF rappresenta un ostacolo significativo per chi manca di competenze tecniche avanzate. Come evidenziato in letteratura, lavorare con strumenti come SPARQL impone vincoli rigorosi[VFQP25] e richiede una familiarità profonda con ontologie complesse e convenzioni di metadati[Vaš24].

Tali approcci si basano su query create manualmente, che risultano fragili e difficili da adattare a esigenze mutevoli[Mar25]. Questo pone il ricercatore di fronte all’ostacolo cognitivo della “scatola vuota”: chi possiede un bisogno informativo vago o in via di definizione viene paralizzato dalla necessità di dichiarare tutto a priori.

2.2.2 Il divario tra ricerca strutturata e ricerca esplorativa

Alla base dei tradizionali sistemi di Information Retrieval (IR) vi è un assunto fondamentale: che l’utente inizi l’interazione con un bisogno informativo perfettamente strutturato. Nel dominio umanistico, tuttavia, l’utente procede per esplorazione.

Questo comportamento è stato teorizzato da Marcia Bates con il modello *berry-picking*[Bat89]: una strategia di ricerca non lineare in cui gli studiosi si muovono iterativamente, aggiustando la rotta e raffinando gli obiettivi man mano che acquisiscono nuove informazioni durante l’esplorazione. In un tale scenario di ricerca esplorativa, assume un valore inestimabile il fenomeno della *serendipity*, la scoperta inattesa di pattern rilevanti durante la navigazione dei dati, che gli strumenti tradizionali non incoraggiano, anzi, spesso ostacolano.

Considerando il netto contrasto tra la rigidità sintattica di SPARQL e la natura fluida ed evolutiva della ricerca umanistica, emerge con chiarezza la necessità di un nuovo paradigma di interazione, che consenta ai ricercatori di esprimere bisogni emergenti senza dover ricorrere a linguaggi formali.

2.2.3 La “scatola vuota” e le tre barriere di SPARQL

Costringere ricercatori umanisti a esprimere bisogni complessi tramite endpoint SPARQL genera un ostacolo su tre livelli:

1. **Barriera cognitiva:** L’interazione esige la scrittura in un linguaggio dichiarativo formale e l’obbligo di conoscere complessi prefissi ontologici (es. `dc:`, `exif:`, `ebucore:`) solo per abbozzare una query di base.
2. **Barriera strutturale:** L’assenza di una mappa visiva delle relazioni all’interno del grafo RDF fa sì che l’utente non sappia quali proprietà esistano realmente nel database né come i dati siano formalizzati nell’ontologia.
3. **Barriera pratica:** La forte rigidità sintattica determina che un singolo errore di battitura produca un set di risultati vuoto. L’assenza di messaggi di errore parlanti impedisce qualsiasi tentativo di debugging autonomo.

Studi sull’usabilità delle interfacce per Knowledge Graphs confermano che linguaggi come SPARQL impongono vincoli rigidi e richiedono una conoscenza esplicita degli schemi, competenze estranee ai professionisti del patrimonio culturale[VFQP25].

2.3 Fondamenti di architettura dell'informazione

La progettazione di un sistema di accesso per un database RDF non può limitarsi alla pura efficienza computazionale del recupero dati. Deve invece rispondere ai principi fondamentali dell'*architettura dell'informazione*, ovvero la disciplina che studia come rendere le informazioni realmente trovabili e usabili.

In un contesto umanistico, costringere l'utente a interfacciarsi con la complessità dello schema sottostante attraverso filtri rigidi o linguaggi formali genera un ostacolo cognitivo. Procediamo ora ad analizzare i principi teorici che giustificano il ricorso a un'interfaccia conversazionale mediata da LLM.

2.3.1 L'illusione della ricerca algoritmica

I sistemi di reperimento basati su query formali (SQL, SPARQL) assumono erroneamente che l'utente sappia sempre cosa cercare e come tradurre il proprio bisogno in un costrutto logico. Peter Morville e Louis Rosenfeld hanno smontato questo assunto come un grave errore concettuale:

“Consideriamo questo modello pericoloso perché basato su un equivoco: che trovare informazioni sia un problema diretto, che può essere risolto da un semplice approccio algoritmico. Dopo tutto, abbiamo risolto la sfida del recupero dati — che, ovviamente sono fatti e cifre — con tecnologie database come SQL. Così procede la teoria, si trattano idee e concetti incorporati nei documenti testuali semistrutturati nello stesso modo”
[MR08]

I sistemi algoritmici tradizionali falliscono nel dominio umanistico perché sono progettati per recuperare *dati esatti*, mentre gli studiosi ricercano *concetti, idee e relazioni*. Tali concetti sono per loro natura ambigui e sfumati. Un intermediario LLM colma esattamente questo divario: funge da traduttore semantico che accetta l'ambiguità e la ricchezza del linguaggio naturale e la converte in modo trasparente nell'approccio algoritmico rigido richiesto dalla macchina.

2.3.2 Recall, Precision e il compromesso ineluttabile

Un problema critico delle interfacce tradizionali a filtri è l'impossibilità di calibrare dinamicamente il raggio della ricerca. Morville e Rosenfeld spiegano che ogni ricerca è un compromesso matematico tra due metriche inversamente correlate:

$$\text{recall} = \frac{|\text{doc rilevanti recuperati}|}{|\text{doc nella raccolta}|} \quad (2.1)$$

$$\text{precision} = \frac{|\text{doc rilevanti recuperati}|}{|\text{doc rilevanti della raccolta}|} \quad (2.2)$$

“[...] Non sarebbe bello poter ottenere contemporaneamente elevati valori di recall e precision? Purtroppo non si può avere botte piena e moglie ubriaca: recall e precision sono inversamente correlati”
[MR08]

In un'interfaccia a maschera con decine di campi e filtri ontologici, l'utente è costretto a dichiarare a priori il livello di precisione desiderato. Se sbaglia un filtro, ottiene zero risultati (eccesso di Precision); se ne omette uno, viene sommerso da migliaia di risultati irrilevanti (eccesso di Recall).

In una conversazione in linguaggio naturale con un LLM, invece, l'aggiustamento avviene in modo iterativo e fluido. L'utente può iniziare con una richiesta esplorativa e ampia (es. “Mostrami tutti i manoscritti del XIV secolo”, privilegiando il Recall) e, osservando i primi risultati, raffinare il tiro conversando (“Filtra solo quelli con miniature e digitalizzati ad alta risoluzione”, aumentando la Precision). L'LLM permette di variare queste metriche dinamicamente senza costringere l'utente a decostruire e riscrivere manualmente una complessa sintassi di query.

2.3.3 Carico cognitivo: la Legge del minimo sforzo e la Legge di Hick

Dal punto di vista del carico cognitivo, esporre all'utente l'immensa topologia di un Knowledge Graph attraverso decine di menu a tendina o l'obbligo di conoscere specifiche ontologie porta alla frustrazione potrebbe sfociare nell'abbandono del sistema.

Luca Rosati analizza questo fenomeno attraverso due principi cognitivi fondamentali. Il primo è la *legge del minimo sforzo*:

“La legge del minimo sforzo (least effort) regola quindi il nostro sistema di information seeking, al punto che spesso accettiamo anche contenuti di bassa qualità o affidabilità (se più facili da ottenere e usare) pur di non ricorrere a strategie di ricerca attive, le quali comportano necessariamente sforzi, tempi e competenze maggiori” [Ros07]

Se l'interfaccia richiede la fatica di apprendere prefissi RDF o di comprendere uno schema dati complesso, è plausibile che l'utente rinuncerà alla ricerca.

Il secondo principio è la *Legge di Hick*, la quale afferma che, in presenza di n alternative equiprobabili, il tempo necessario per selezionarne una è proporzionale (a meno di una costante additiva) al logaritmo in base 2 del numero di scelte più 1:

$$\text{tempo} = a + b \log_2(n + 1)$$

I coefficienti a e b dipendono da specifiche condizioni al contorno, tra cui le modalità di presentazione delle opzioni e il grado di familiarità dell'utente [Ros07].

Tuttavia, Rosati sottolinea che, qualora le alternative “siano presentate in modo confuso” o costituiscano “una lista disordinata” priva di un criterio logico riconoscibile, il tempo di decisione scala in modo lineare, aumentando il carico cognitivo fino a divenire insostenibile e causando paralisi decisionale. Esporre l'utente a un form SPARQL nudo o a una maschera di ricerca avanzata con decine di menu a tendina equivale esattamente a questo: innescare un “paradosso della scelta” paralizzante.

Un'interfaccia conversazionale basata su LLM nasconde questa complessità strutturale nel backend. L'utente si trova di fronte a un singolo "spazio" (un box di chat), azzerando l'effetto della Legge di Hick e assecondando la Legge del minimo sforzo. Non dovendo scegliere tra decine di opzioni incomprensibili, il ricercatore delega il carico cognitivo della traduzione ontologica all'LLM, dedicando le proprie energie intellettuali esclusivamente all'analisi e alla scoperta delle risorse.

2.3.4 Paradigma classico: navigazione e filtri rigidi

L'architettura dell'informazione si definisce come il design strutturale di ambienti informativi finalizzato a plasmare l'usabilità e la trovabilità[MR08]. Poggia su quattro sistemi interconnessi:

- (i) sistemi di organizzazione, che categorizzano le informazioni;
- (ii) sistemi di etichettatura, che ne determinano la rappresentazione linguistica;
- (iii) sistemi di navigazione, che consentono l'orientamento nello spazio;
- (iv) sistemi di ricerca, che permettono interrogazioni dirette.

Applicato al Datavault, l'interrogazione tradizionale mediante filtri rigidi o SPARQL presenta tre limiti strutturali:

1. Pretende conoscenza a priori di stringhe esatte, incompatibile con la ricerca umanistica [Vaš24];
2. Non gestisce sinonimia, sfumature e vaghezza del linguaggio naturale;
3. Obbliga la formulazione di query multi-criterio in sintassi rigide.

Queste rigidità abbattano la "trovabilità": l'informazione esiste fisicamente ma rimane irraggiungibile per l'utente.

2.3.5 Il nuovo paradigma: ricerca semantica con LLM

L'introduzione di un LLM modifica radicalmente il paradigma di esplorazione dei dati semantici, agendo come un intermediario e traduttore capace di convertire fluidamente il linguaggio naturale in interrogazioni strutturate come query SPARQL.

Questa trasformazione definisce un concetto chiave nell'interazione uomo-dato: non è più il ricercatore a dover imparare il linguaggio della macchina, ma è il sistema stesso che si adatta all'utente, comprendendone le richieste nel formato linguistico naturale. Ne consegue che questo approccio sposta l'intero carico di complessità cognitiva e sintattica dall'utente al sistema computazionale.

Questo trasferimento della complessità si lega esplicitamente al concetto di "findability" (trovabilità)[Ros07]. Lo scopo del nuovo sistema non è semplicemente rendere una risorsa formalmente accessibile, ma garantire che essa risulti effettivamente trovabile dal ricercatore nel momento del bisogno.

La ricerca semantica mediata dall'LLM aumenta in modo decisivo la trovabilità per tre ragioni:

- abbassa drasticamente la soglia di accesso all'informazione, permettendo a chiunque di fare una domanda diretta al database;
- il modello gestisce in modo nativo la vaghezza, l'ambiguità e i sinonimi intrinseci nel linguaggio umano;
- consente di esprimere bisogni informativi multi-criterio complessi, impossibili da supportare attraverso form a tendina.

Mantenendo un doveroso grado di obiettività scientifica, occorre tuttavia dichiarare che il paradigma basato su LLM presenta sfide intrinseche: il rischio di allucinazioni (invenzione di predicati inesistenti), la dipendenza dall'accuratezza del prompt di sistema, e l'elevato costo computazionale delle chiamate ai modelli generativi. Queste metriche verranno analizzate in dettaglio nel capitolo 5.

2.4 Retrieval-Augmented Generation (RAG) come soluzione tecnologica

2.4.1 Il paradigma RAG tradizionale

Retrieval-Augmented Generation (RAG) è un paradigma che combina la potenza dei modelli generativi con l'affidabilità del recupero di informazioni da fonti strutturate. La struttura di base è semplice ma efficace:

1. **Ingresso:** l'utente pone una domanda in linguaggio naturale;
2. **Retrieval:** un modulo di ricerca estrae documenti o frammenti rilevanti da una base di conoscenza strutturata;
3. **Augmentation:** il contesto recuperato viene fornito al modello generativo come "prompt esteso";
4. **Generation:** il modello generativo produce una risposta grounded nei documenti recuperati, riducendo le allucinazioni.

Questo paradigma riduce significativamente il rischio di allucinazioni perché il modello è vincolato a operare all'interno di uno spazio di conoscenza ben definito.

2.4.2 RAG applicato a Knowledge Graphs

Nel nostro caso, il "corpus" da cui recuperare informazioni è uno **schema RDF strutturato**. L'applicazione di RAG a Knowledge Graphs richiede una variante: anziché recuperare documenti di testo, recuperiamo *frammenti di schema, esempi di predicati, e descrizioni di ontologie*.

Quindi il flusso diventa:

1. **Ingresso:** domanda dell'utente (es. "Mostrami tutte le immagini digitalizzate con una Nikon nel 2022");
2. **Retrieval:** il sistema accede allo schema RDF del Datavault e recupera i predicati rilevanti (es. `exif:make`, `exif:dateTime`);
3. **Augmentation:** lo schema e gli esempi di predicati vengono forniti al modello LLM come parte del prompt di sistema;
4. **Generation:** il modello LLM produce una query SPARQL coerente con lo schema;
5. **Esecuzione:** la query viene eseguita su Blazegraph e i risultati vengono restituiti all'utente.

Questo approccio combina i lati positivi di entrambi: la flessibilità del linguaggio naturale e l'affidabilità di una fonte di conoscenza strutturata e verificabile.

2.4.3 Fine-tuning e LoRA per Domain Adaptation

Un miglioramento ulteriore consiste nel fine-tuning del modello LLM su un dataset specifico del dominio. Anziché usare un LLM generico, possiamo adattare un modello più piccolo (es. `Phi-3-mini`) a generare query SPARQL corrette per il Datavault specifico.

Tecniche come LoRA (*Low-Rank Adaptation*) permettono un fine-tuning efficiente senza memorizzare l'intero modello in memoria. Il modello apprende:

1. La sintassi SPARQL corretta per il Datavault;
2. L'ontologia specifica (predicati, tipi di dato, convenzioni di naming);
3. Strategie di generazione robuste (evitare predicati inesistenti, gestire date, etc.);
4. Quando e come segnalare query out-of-domain.

Questo approccio riduce significativamente il rischio di allucinazioni e aumenta l'accuratezza della generazione di query.

2.5 Conclusioni del capitolo

In questo capitolo abbiamo analizzato il contesto scientifico e tecnologico che motiva il nostro contributo. Abbiamo visto che:

1. Gli approcci tradizionali (SPARQL nudo, filtri rigidi) falliscono nel dominio umanistico perché impongono barriere cognitive, strutturali e pratiche;
2. I modelli teorici dell'architettura dell'informazione (berrypicking, precision e recall, carico cognitivo) giustificano il ricorso a un'interfaccia conversazionale mediata da LLM;

3. Le tecnologie emergenti (RAG, fine-tuning con LoRA) rendono possibile un'implementazione affidabile e accurata di tale interfaccia.

Avendo chiarito il fondamento scientifico e tecnologico della nostra soluzione, il capitolo 3 presenta nel concreto il nostro LLM Query Builder: il dominio di applicazione (DH.ARC e il Datavault), il profilo degli utenti target, l'architettura funzionale del sistema, e i casi d'uso che dimostrano il valore aggiunto della nostra proposta nel contesto umanistico.

Capitolo 3

Il prototipo

3.1 Introduzione al capitolo

Avendo chiarito il fondamento scientifico e tecnologico nel capitolo 2, presentiamo ora il prototipo di LLM Query Builder sviluppato per sopperire alle problematiche riscontrate.

Questo capitolo è organizzato secondo una prospettiva *user-centered*: partiamo dal dominio di applicazione (DH.ARC e il Datavault) e dal profilo degli utenti target (paragrafo 3.2), passiamo all’architettura funzionale e all’interfaccia del sistema (paragrafo 3.3), e concludiamo con tre scenari d’uso concreti (paragrafo 3.4) arricchiti dalle relative schermate, per dimostrare il valore aggiunto della nostra proposta.

3.2 Il dominio e gli utenti target

3.2.1 Il contesto operativo: DH.ARC e il Datavault

Il Digital Humanities Advanced Research Centre (DH.ARC) è un polo di ricerca dell’Università di Bologna afferente al Dipartimento di Filologia Classica e Italianistica (FICLIT), nato con l’obiettivo di “connettere studenti, ricercatori, personale IT e professori del FICLIT e del Dipartimento di Informatica - Scienza e Ingegneria (DISI)”[ALM26]. Il centro ha lo scopo di fungere da hub per gli studiosi e le istituzioni, supportandoli “nella progettazione, sviluppo e manutenzione di progetti di ricerca DH” e promuovendo iniziative innovative nel campo umanistico.

All’interno di questo ecosistema, una delle infrastrutture tecnologiche di conservazione è il *Datavault*, un repository digitale basato sull’architettura Fedora. Questo sistema svolge un ruolo cruciale nel ciclo di vita degli asset digitali culturali, poiché è stato appositamente progettato per supportare l’archiviazione a lungo termine, la conservazione e la gestione strutturata delle risorse digitalizzate. Dal punto di vista tecnico, il repository espone le proprie risorse sotto forma di triple RDF per favorire l’interoperabilità semantica e l’integrazione con strumenti di analisi avanzata.[FD25] Tuttavia, il Datavault ospita attualmente grande mole di dati, dell’ordine di milioni di

triple RDF fortemente interconnesse e strutturalmente molto simili tra loro. Questa vastità e omogeneità sintattica rende la ricerca, l'estrazione e il recupero di specifiche risorse un'operazione complessa e non immediata, ponendo sfide significative per l'esplorazione del contenuto.

3.2.2 Profilo dell'utente finale: i ricercatori del centro

Profilo disciplinare e competenze tecniche

Gli utenti di riferimento del centro DH.ARC sono docenti, ricercatori, assegnisti e studenti di discipline prettamente umanistiche, quali filologia, storia, storia dell'arte e archeologia.[PT22] Questi profili hanno profonde competenze nel dominio delle fonti culturali e nell'analisi dei testi critici, ma tendono ad essere privi di un background tecnico/informatico.

La loro alfabetizzazione digitale nel campo dell'Information Retrieval si limita generalmente all'impiego di strumenti "pronti all'uso": cataloghi OPAC, database relazionali con maschere pre-configurate o motori di ricerca generici; raramente hanno familiarità con triplestore RDF, SPARQL, o Knowledge Graphs.

Caratterizzazione dei bisogni informativi

In questo ecosistema, gli utenti non ricercano identificativi esatti o risposte chiuse; piuttosto, adottano comportamenti di ricerca esplorativa, procedendo per esplorazione libera cercando di collegare concetti e scoprire associazioni non immediate tra i documenti. Interrogando il Datavault, gli umanisti del centro necessitano di porre interrogazioni cross-dominio articolate, formulando mentalmente domande come:

| *"Quali immagini di codici medievali del XIV secolo sono state digitalizzate nel corso del progetto X?"*

o ancora:

| *"Mostrami tutti i manoscritti attribuiti a un determinato autore e digitalizzati nell'ultimo anno".*

3.3 Architettura Funzionale e Interfaccia Utente

3.3.1 L'interfaccia: come si usa il Data Vault Explorer

Per risolvere i problemi di carico cognitivo e rigidità formale descritti nel capitolo precedente, il sistema si presenta all'utente attraverso un'interfaccia minimalista e conversazionale (il Data Vault Explorer). Per superare le barriere legate all'alto carico cognitivo e alla rigidità formale descritte nelle sezioni precedenti, l'applicativo è stato esteso introducendo un modulo di interazione minimalista e conversazionale.

Invece di dover navigare attraverso complessi moduli a filtri multipli o di dover studiare a priori l'ontologia sottostante per formulare interrogazioni dirette, l'utente ha ora a disposizione un singolo e intuitivo punto di ingresso: la *LLM Search Bar*. Questa scelta di design permette di aggirare la complessità del database dietro

un'interfaccia familiare. Il sistema si presenta visivamente in modo simile ai motori di ricerca, tuttavia, è potenziato dalla capacità del Large Language Model di interpretare le sfumature del linguaggio naturale, traducendo in tempo reale l'intento esplorativo del ricercatore in una query SPARQL formale ed eseguibile.

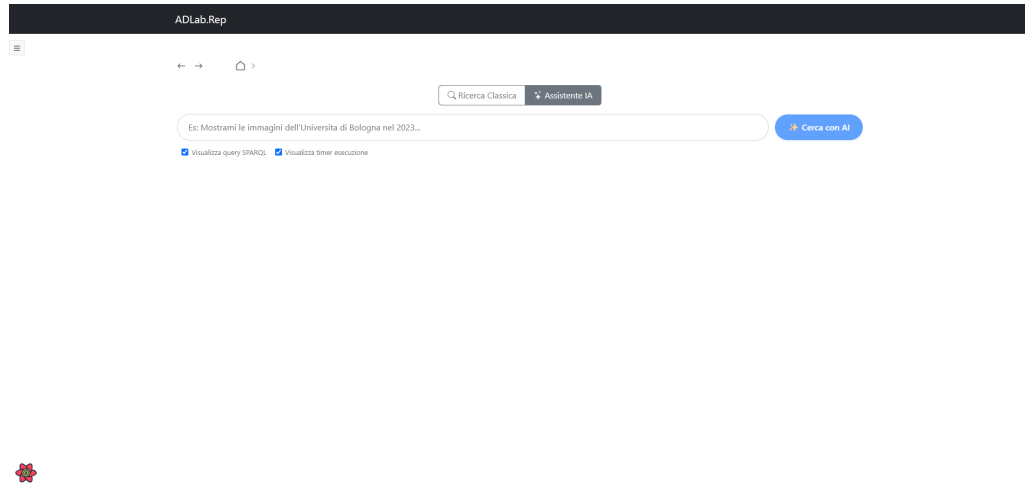


Figura 3.1: L'interfaccia principale del Data Vault Explorer con la LLM Search Bar.

Come visibile in figura 3.1, l'utilizzo del sistema si riduce a un'azione elementare: digitare il proprio bisogno informativo in italiano naturale. Il sistema restituisce i risultati strutturandoli in una lista di risorse, permettendo un'esplorazione fluida e intuitiva dei dati.

3.3.2 Il flusso dell'interazione

Dal punto di vista dell'utente, l'interazione con il sistema si articola secondo un flusso invisibile ma rigorosamente strutturato, progettato per astrarre completamente la complessità tecnica sottostante. Questo processo è schematizzato nella Figura 3.2.

Come illustrato nello schema, le fasi del processo sono le seguenti:

1. **Input:** l'utente digita il proprio bisogno informativo in italiano in forma libera. In questa fase non vi sono campi strutturati da compilare né regole di sintassi da rispettare.
2. **Elaborazione:** il testo naturale viene inviato al componente intelligente (il modello LLM fine-tuned) che, essendo a conoscenza dello schema RDF e dell'ontologia del Datavault, produce autonomamente la query SPARQL corrispondente.
3. **Esecuzione e Output:** la query SPARQL viene inoltrata a Blazegraph. L'utente non è esposto al codice formale (salvo esplicita richiesta), ma visualizza esclusivamente i risultati pertinenti tradotti nell'interfaccia grafica, aggirando così del tutto la barriera tecnologica.

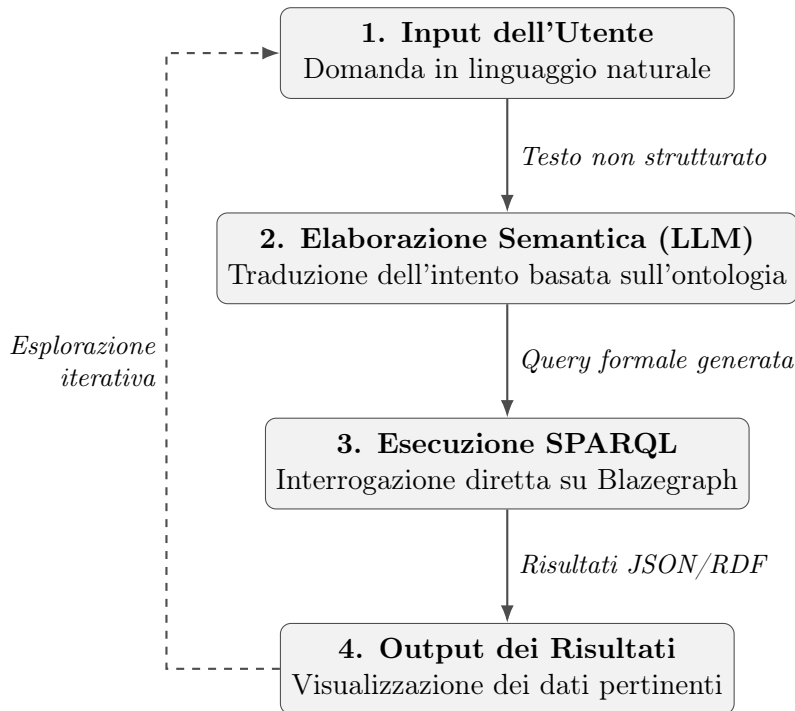


Figura 3.2: Schema del flusso di interazione tra utente, LLM Query Builder e Datavault.

3.3.3 I componenti del sistema e le loro responsabilità

Il sistema si articola in tre componenti principali:

1. **Frontend (Data Vault Explorer):** raccoglie l'input testuale e renderizza i risultati in formato navigabile.
2. **Backend (LLM Query Builder + RAG Engine):** il “cervello” del sistema. Riceve la domanda, recupera lo schema RDF, genera la query SPARQL tramite il modello e gestisce gli errori semantici (evitando le allucinazioni).
3. **Data Layer (Blazegraph + Schema RDF):** immagazzina i milioni di triple ed esegue l'interrogazione generata dal backend.

3.4 Casi d'uso principali ed esempi

Descriviamo tre scenari d'uso concreti che dimostrano l'efficacia del sistema presso DH.ARC, evidenziando il parallelismo tra l'interazione logica (traduzione LLM e sanitizzazione) e il riscontro visivo restituito all'utente.

3.4.1 Caso d'uso 1: ricerca per caratteristiche di acquisizione e gestione anomalie

Attore: un ricercatore che vuole individuare immagini acquisite con specifiche caratteristiche tecniche.

Input testuale:

“Filtra i risultati mostrandomi le risorse
acquisite con il dispositivo NextScan”

```

1 PREFIX exif: <http://www3.org/2003/12/
  exif/ns#>
2 PREFIX fedora: <http://fedora.info/
  definitions/v4/repository#>
3 PREFIX ebucore: <http://www.ebu.ch/
  metadata/ontologies/ebucore/ebucore
  #>
4 PREFIX ldp: <http://www.w3.org/ns/ldp#>
5 PREFIX dc: <http://purl.org/dc/elements
  /1.1/>
6
7 SELECT DISTINCT ?image WHERE {
8   ?image exif:make ?make .
9   FILTER(CONTAINS(LCASE(?make), "nextscan
    "))
10 }
11 LIMIT 20
12

```

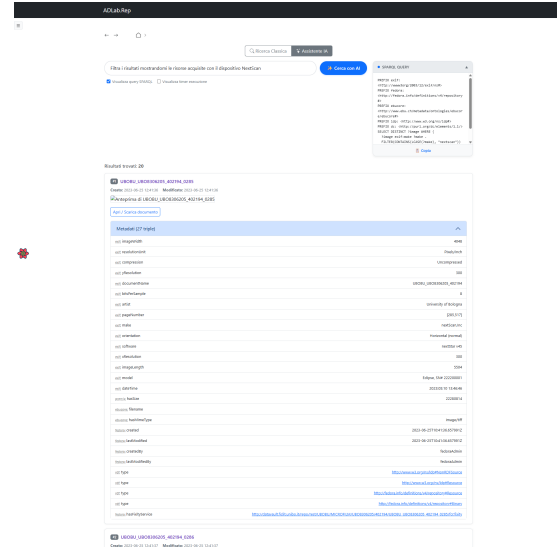


Figura 3.3: Caso d’uso 1: A sinistra la query SPARQL generata e sanitizzata, a destra l’estrazione delle risorse corrispondenti al dispositivo.

Risoluzione del problema: Oltre a delegare all’LLM l’onere della traduzione del linguaggio naturale nel predicato corretto (`exif:make`), questo scenario dimostra la resilienza dell’architettura.

3.4.2 Caso d’uso 2: disambiguazione semantica dell’autore

Attore: un responsabile di un progetto di digitalizzazione o ricercatore interessato alla provenienza materiale dello scatto.

Input testuale:

“Filtra le risorse mostrandomi i file in cui
l’autore dello scatto è University”

```

1 PREFIX exif: <http://www3.org/2003/12/
  exif/ns#>
2 # [...]
3
4 SELECT DISTINCT ?image WHERE {
5   ?image exif:artist ?artist .
6   FILTER(CONTAINS(LCASE(?artist), "
    university"))
7 }
8 LIMIT 20
9

```

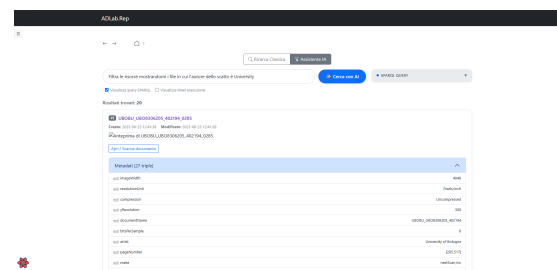


Figura 3.4: Caso d’uso 2: Disambiguazione semantica dell’autore e visualizzazione dei risultati.

Risoluzione del problema: Questo scenario evidenzia la capacità dell’LLM di operare una corretta disambiguazione semantica. Il modello comprende l’intento

dell'utente distinguendo l'autore materiale dello scatto (`exif:artist`) dall'utente responsabile del caricamento nel Data Vault (`fedora:createdBy`).

3.4.3 Caso d'uso 3: ricerca multi-criterio con filtro temporale

Attore: un ricercatore che vuole analizzare la produzione digitale filtrandola all'interno di un preciso arco temporale.

Input testuale:

“Mostrami tutte le risorse comprese tra l'anno 2023 e il 2026”

```
1 PREFIX exif: <http://www3.org/2003/12/
  exif/ns#>
2 # [...]
3
4 SELECT DISTINCT ?image WHERE {
5 ?image exif:dateTime ?date .
6 FILTER(STR(?date) >= "2023" && STR(?date)
7 ) <= "2026")
8 }
9 LIMIT 20
```

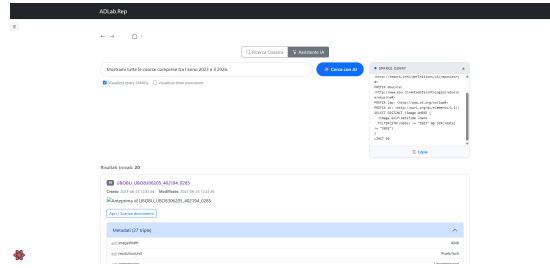


Figura 3.5: Caso d'uso 3: Applicazione logica del range temporale ed estrazione delle immagini.

Risoluzione del problema: L'estrazione basata su range temporali richiede una sintassi SPARQL rigorosa e insidiosa per un utente non tecnico. Il sistema individua automaticamente il predicato `exif:dateTime` e costruisce i costrutti logici di comparazione testuale (`>=` e `<=`).

3.5 Conclusioni del capitolo

In questo capitolo abbiamo presentato il prototipo e la sua interfaccia, rispondendo attivamente alle criticità sollevate nel capitolo 2:

1. Il problema della “**scatola vuota**” e la paralisi decisionale (Legge di Hick) sono stati superati adottando un'interfaccia basata unicamente sull'input testuale naturale.
2. L'obbligo di conoscere schemi complessi (es. prefissi EXIF o Dublin Core) è stato demandato al backend, azzerando il carico cognitivo per l'utente finale.
3. La rigidità formale di SPARQL è stata assorbita dall'LLM, che agisce come mediatore invisibile.

Il capitolo 4 approfondirà il dietro le quinte tecnologico: la struttura del backend, l'ottimizzazione del proxy Vite e il delicato processo di fine-tuning del modello linguistico.

Capitolo 4

Architettura e dettagli implementativi

4.1 Introduzione al capitolo

In questo capitolo viene illustrata nel dettaglio l'architettura tecnica e le scelte implementative che costituiscono il motore dell'LLM Query Builder. Se il capitolo 3 ha descritto il sistema dal punto di vista dell'utente finale focalizzandosi sulla transizione da domanda naturale a risultati navigabili, in questo capitolo verranno analizzati i componenti software, lo stack tecnologico e le metodologie di addestramento che rendono possibile tale comportamento.

Il capitolo segue il flusso logico della costruzione del sistema: si parte dall'infrastruttura di base e dalle scelte tecnologiche complessive (paragrafo 4.2), passando alla pipeline di fine-tuning del modello (paragrafo 4.3), quindi si analizza l'architettura del backend e l'integrazione con il frontend (paragrafo 4.4), per concludere con il formato e la struttura del dataset di addestramento (paragrafo 4.5).

4.2 Stack tecnologico: Fedora e Blazegraph

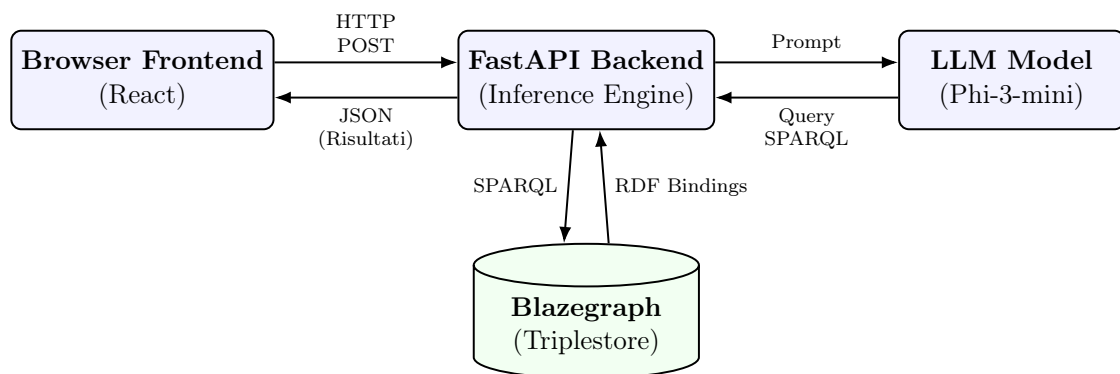


Figura 4.1: Architettura logica e flusso di comunicazione del sistema.

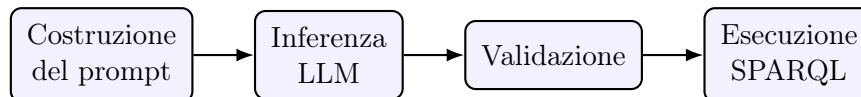
L'infrastruttura dati su cui si appoggia l'LLM Query Builder è costituita da Fedora e Blazegraph. Blazegraph, in particolare, opera come triplestore ad alte prestazioni

per il Datavault, fornendo l’endpoint SPARQL interrogato dal nostro sistema. Il lavoro è consistito nel creare un layer intermedio capace di dialogare fluidamente con Blazegraph, astraendo la sintassi del linguaggio SPARQL dall’interfaccia esposta al ricercatore.

4.2.1 I tre layer del sistema

A livello logico, il sistema suddivide il flusso di lavoro in tre layer:

- **Presentazione (Datavault Browser):** applicazione React con due barre di ricerca testuale, una per una ricerca google-like `<SearchField/>` e una per una ricerca avanzata con il tool `<LLMSearchBar/>` selezionabili da un bottone centrato sopra di esse. In modalità “Assistente IA”, è possibile la visualizzazione della query SPARQL generata per utenti più esperti. Raccoglie l’input testuale dell’utente e lo invia al backend.
- **Applicativo (LLM Query Builder backend, FastAPI):** il “cervello” del sistema che riceve la domanda naturale, dirige il flusso:



e gestisce gli errori (query malformate, allucinazioni, timeout, richieste out-of-domain) e restituisce risultati strutturati al frontend.

- **LDati (Datavault / Blazegraph):** immagazzina i milioni di triple RDF del Datavault, esegue la query SPARQL ricevuta dal backend e ne restituisce i risultati grezzi.

4.2.2 Il flusso end-to-end di una richiesta

Una richiesta tipica attraversa il sistema secondo i seguenti passi:

1. L’utente digita una domanda in linguaggio naturale (es. “Mostrami le immagini scansionate con una Nikon nel 2022”) nella barra di ricerca del frontend.
2. Il frontend invia un `HTTP POST` al backend con il testo della domanda.
3. Il backend costruisce il prompt completo (system prompt con le regole di generazione + domanda dell’utente, si veda paragrafo 4.5.1) e lo sottopone al modello LLM.
4. Il modello genera una query SPARQL (o un JSON di errore, se la richiesta è fuori dominio).
5. Il backend ripulisce e valida l’output (si veda il “sanitizzatore”, paragrafo 4.4.1).
6. Se valida, la query viene eseguita su Blazegraph.
7. I risultati vengono trasformati in struttura ad albero e restituiti al frontend, che li visualizza in forma navigabile.

4.2.3 Scelte tecnologiche principali

Il sistema combina le seguenti tecnologie, ciascuna scelta per ragioni specifiche legate ai vincoli del progetto:

- **React** (frontend): per la parte di presentazione, già descritta funzionalmente nel capitolo 3.
- **FastAPI** (backend): scelto per la type safety offerta da Pydantic, la documentazione automatica, le performance asincrone e la facilità di integrazione diretta con PyTorch/Transformers per servire il modello.
- **Phi-3-mini** (modello LLM base): dimensione contenuta (3.8 miliardi di parametri) che rende fattibile il fine-tuning mantenendo buone performance di base su compiti di generazione strutturata.
- **Unsloth + LoRA** (fine-tuning): Unsloth velocizza il fine-tuning e riduce il consumo di memoria; LoRA (Low-Rank Adaptation) permette di adattare il modello aggiornando solo un numero ridotto di parametri, evitando i costi proibitivi del full fine-tuning.
- **Blazegraph** (triplestore): tecnologia già in uso presso DH.ARC per il Datavault, con supporto nativo a SPARQL 1.1 e scalabilità fino a centinaia di milioni di triple.

4.3 Pipeline di fine-tuning dell'LLM

Per ottenere traduzioni accurate dal linguaggio naturale a SPARQL, l'utilizzo di un LLM generico si è rivelato insufficiente: un modello addestrato su testo web conosce SPARQL in linea di principio, ma non è ottimizzato per generare query corrette per lo specifico schema del Datavault, rischiando di allucinare predicati inesistenti o di non comprendere le particolarità dell'ontologia reale. Si è resa quindi necessaria una fase di *domain adaptation* tramite fine-tuning.

4.3.1 Dataset generativo e schema SPARQL

L'addestramento è stato basato su un dataset generativo, prodotto tramite lo script `genera_dataset.py` secondo un approccio a **template parametrici**: per ciascuna categoria di query viene definita una funzione Python che, dati alcuni parametri (un anno, un nome di dispositivo, un formato file, ecc.), restituisce la query SPARQL corrispondente già popolata con quei valori. La domanda in linguaggio naturale che accompagna ciascuna query viene generata in parallelo variando lessico e struttura della frase, in modo da esporre il modello a una pluralità di formulazioni per lo stesso intento.

Il ciclo di generazione

Il cuore del generatore è il ciclo di costruzione del dataset, riportato (in forma semplificata) di seguito:

```

1 def crea_dataset_massivo(target: int = 10000):
2     # ... inizializzazione contatori ...
3     tipi_sparql = list(range(1, 14))
4
5     while conteggio < target:
6         # Iniezione controllata di esempi Fuori Dominio
7         forza_ood = (ood_count < ood_target) and (
8             sparql_count >= sparql_target or random.random() < 0.15
9         )
10        if forza_ood:
11            frase = random.choice(out_of_domain)
12            query = '{"error": "OUT_OF_DOMAIN"}'
13            ood_count += 1
14        else:
15            # Estrazione di una categoria e popolamento del template
16            tipo = random.choice(tipi_sparql)
17            if tipo == 1:
18                ... # popolamento template categoria 1
19            elif tipo == 2:
20                ... # popolamento template categoria 2
21            # [...]
22            else: # tipo == 13
23                ... # popolamento template categoria 13
24
25        # Salvataggio in formato JSONL pronto per il fine-tuning,
26        # con lo stesso system prompt usato in inferenza
27        riga_dataset = {
28            "messages": [
29                {"role": "system", "content": SYSTEM_PROMPT},
30                {"role": "user", "content": frase},
31                {"role": "assistant", "content": query},
32            ]
33        }

```

Code 4.1: Struttura del ciclo di generazione del dataset

Questa struttura garantisce due proprietà desiderabili per il dataset: una distribuzione controllata tra le diverse categorie di query (evitando lo sbilanciamento verso le categorie più semplici da generare), e un'iniezione mirata di esempi out-of-domain in proporzione fissa (circa il 10% del totale), necessaria affinché il modello impari a riconoscere e rifiutare richieste estranee al dominio del Datavault senza che questo comportamento venga sotto-rappresentato nel training. Inoltre, l'approccio generativo ha man mano permesso di raffinare gli esempi, correggendo a monte gli errori sistematici nella gestione dei prefissi ontologici e dei metadati emersi durante le prime fasi di test del modello.

Le 13 categorie di query SPARQL (+1 Out-of-Domain)

La versione finale del generatore comprende 13 funzioni template, ciascuna corrispondente a un intento di ricerca distinto. Le funzioni sono state progettate per coprire lo spettro delle interrogazioni tipiche del dominio umanistico, più la gestione separata delle query fuori dominio:

#	Funzione	Categoria
1	<code>query_autore_anno</code>	Ricerca combinata: autore dello scatto + anno
2	<code>query_dispositivo</code>	Ricerca per dispositivo di acquisizione (<code>exif:make</code>)
3	<code>query_anno</code>	Ricerca generica per anno
4	<code>GOLDEN_QUERY_LIBERA</code>	Richiesta dell'intero archivio ("mostrami tutto")
5	<code>query_container</code>	Ricerca in una cartella/container (<code>ldp:contains+</code>)
6	<code>query_testo</code>	Ricerca full-text su nome file (<code>ebucore:filename</code>)
7	<code>query_formato</code>	Filtro per formato/MIME type (es. <code>image/jpeg</code>)
8	<code>query_data_esatta</code>	Ricerca per data precisa (anno, mese, giorno)
9	<code>query_intervallo_anni</code>	Ricerca per intervallo temporale (range di anni)
10	<code>query_not_anno</code>	Negazione: esclusione di un anno specifico
11	<code>query_or_formati</code>	OR logico tra due formati file
12	<code>query_limite_custom</code>	Paginazione: restituzione degli ultimi N file
13	<code>query_uploader</code>	Ricerca per autore del caricamento (<code>fedora:createdBy</code>)
—	<i>(separato)</i>	Out-of-Domain: richieste estranee al dominio (~10% del dataset)

Tabella 4.1: Le 13 categorie di query SPARQL generate da `genera_dataset.py`, più la categoria Out-of-Domain gestita a parte nel ciclo di generazione.

Si noti come le categorie 1 e 13 condividano lo stesso intento superficiale ("ricerca per autore") ma si distinguano per il predicato RDF target (`exif:artist` per l'autore dello scatto, `fedora:createdBy` per chi ha effettuato il caricamento): questa distinzione, esplicitata anche nella Regola 2 del system prompt (paragrafo 4.5.1), rappresenta uno degli edge case linguistici più rilevanti affrontati nella progettazione del dataset.

4.3.2 Configurazione dell'addestramento con Unsloth

Per ottimizzare i tempi di addestramento e ridurre l'impronta in memoria VRAM, la pipeline è stata implementata avvalendosi della libreria Unsloth, che fornisce un'integrazione nativa e ottimizzata per il fine-tuning con LoRA (Low-Rank Adaptation) su modelli come Phi-3.

L'algoritmo LoRA

LoRA evita i costi proibitivi di un full fine-tuning (che richiederebbe risorse GPU non disponibili sull'hardware consumer usato per il training) congelando i pesi originali del modello e introducendo, per ciascuna matrice di pesi W presa di mira, un aggiornamento di basso rango: $W' = W + \frac{\alpha}{r}BA$ dove $B \in \mathbb{R}^{d \times r}$ e $A \in \mathbb{R}^{r \times k}$, con il rank $r \ll \min(d, k)$. Durante il training vengono aggiornate solamente le matrici A e B , riducendo il numero di parametri allenabili rispetto ai miliardi di parametri del modello completo.

Nel nostro caso $r = 16 = \alpha$ (quindi $\alpha/r = 1$, nessuno scaling aggiuntivo dell'aggiornamento), applicato sia ai proiettori di attention (`q/k/v/o_proj`) sia ai proiettori del blocco feed-forward (`gate/up/down_proj`): una scelta più estensiva rispetto a una LoRA limitata alla sola attention, che aumenta la capacità del modello di adattarsi al vocabolario specifico del dominio (predicati RDF, sintassi SPARQL) a fronte di un numero di parametri allenabili comunque contenuto grazie al rank ridotto.

Iperparametri utilizzati

Parametro	Valore
Modello base	unsloth/Phi-3-mini-4k-instruct
Configurazione LoRA	
LoRA rank (r)	16
LoRA alpha	16
LoRA dropout	0
Target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Bias	none
Gradient checkpointing	True
Random state	3407

Tabella 4.2: Iperparametri di fine-tuning LoRA (1/2): modello base e configurazione dell'adapter LoRA, configurato tramite Unsloth.

Parametro	Valore
Batch size (per device)	1
Gradient accumulation steps	8
Batch size effettivo	8 (1×8)
Warmup steps	10
Epochs	1
Learning rate	2e-4
Optimizer	adamw_8bit
Weight decay	0.01
LR scheduler	linear
Seed	3407
fp16 / bf16	False / False
Packing	False
Dataset num proc	2
Max sequence length	1024 token
Step totali eseguiti	1188
Loss finale	0.010

Tabella 4.3: Iperparametri di fine-tuning LoRA (2/2): configurazione del training tramite `SFTTrainer` (libreria TRL).

Il valore di `max_seq_length` è stato fissato a 1024 token: sufficiente a contenere il system prompt (regole di generazione), la domanda dell'utente in linguaggio naturale, e la query SPARQL attesa, mantenendo comunque un footprint di memoria contenuto durante il training su GPU con VRAM limitata.

Il codice di configurazione effettivamente utilizzato è riportato di seguito. Il primo passo è il caricamento del modello base tramite Unsloth:

```

1 import torch
2 from unsloth import FastLanguageModel
3
4 max_seq_length = 1024
5 dtype = torch.float16
6 load_in_4bit = False
7
8 model, tokenizer = FastLanguageModel.from_pretrained(
9     model_name = "unsloth/Phi-3-mini-4k-instruct",
10     max_seq_length = max_seq_length,
11     dtype = dtype,
12     load_in_4bit = load_in_4bit,
13 )

```

Code 4.2: Caricamento del modello base

Da notare che il modello viene caricato in precisione `float16` e senza quantizzazione a 4 bit: questo è distinto dai flag `fp16` / `bf16` del `SFTTrainer` (entrambi `False`), che controllano invece l'uso di *mixed precision training* (autocast durante il calcolo del gradiente) e non la precisione nativa dei pesi del modello.

Segue la configurazione dell'adapter LoRA e del training vero e proprio:

```
1 model = FastLanguageModel.get_peft_model(  
2     model,  
3     r = 16,  
4     target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",  
5                       "gate_proj", "up_proj", "down_proj"],  
6     lora_alpha = 16, lora_dropout = 0,  
7     bias = "none", use_gradient_checkpointing = True,  
8     random_state = 3407,  
9 )
```

Code 4.3: Configurazione dell'adapter LoRA tramite Unsloth

```
1 from trl import SFTTrainer, SFTConfig  
2 trainer = SFTTrainer(  
3     model = model, tokenizer = tokenizer,  
4     train_dataset = train_dataset,  
5     eval_dataset = eval_dataset,  
6     args = SFTConfig(  
7         per_device_train_batch_size = 1,  
8         gradient_accumulation_steps = 8,  
9         warmup_steps = 10,  
10        num_train_epochs = 1,  
11        learning_rate = 2e-4,  
12        fp16 = False, bf16 = False,  
13        logging_steps = 10,  
14        eval_strategy = "steps",  
15        eval_steps = 50,  
16        save_strategy = "steps",  
17        save_steps = 100,  
18        save_total_limit = 2,  
19        report_to = "none",  
20        optim = "adamw_8bit",  
21        weight_decay = 0.01,  
22        lr_scheduler_type = "linear",  
23        seed = 3407,  
24        output_dir = "outputs",  
25        dataset_text_field = "text",  
26        max_seq_length = max_seq_length,  
27        dataset_num_proc = 2,  
28        packing = False,  
29    ),  
30 )
```

Code 4.4: Configurazione del training tramite SFTTrainer (libreria TRL)

4.4 Architettura del backend e integrazione frontend

4.4.1 Inference engine (FastAPI)

Il coordinamento delle richieste è affidato a un backend sviluppato in Python tramite FastAPI. L'inference engine è stato calibrato per garantire la consistenza del *system prompt* a ogni invocazione, vincolando il modello a mantenere il ruolo di traduttore tecnico. È stata posta particolare attenzione alla corretta generazione del token EOS (*End Of Sentence*), fondamentale per troncare l'output del modello ed evitare allucinazioni aggiuntive in coda alla query SPARQL.

Caricamento del modello e fusione dell'adapter LoRA

A differenza del notebook di training (dove il modello viene caricato tramite `FastLanguageModel` di Unsloth), il backend di inferenza carica il modello tramite le primitive standard di Hugging Face Transformers e PEFT, eseguendo il caricamento una sola volta all'avvio del server e non ad ogni richiesta, per evitare il costo proibitivo di ricaricare e fondere il modello a ogni chiamata:

```
1 @app.on_event("startup")
2 async def load_model():
3     global model, tokenizer
4     print("Inizializzazione del motore LLM...")
5
6     tokenizer = AutoTokenizer.from_pretrained(ADAPTER_PATH)
7
8     # Usiamo la versione base per evitare conflitti di
9     # vocabolario
10    base_model = AutoModelForCausalLM.from_pretrained(
11        "unsloth/Phi-3-mini-4k-instruct",
12        torch_dtype=torch.float32,
13        device_map="cpu"
14    )
15
16    # Fondiamo i pesi LoRA addestrati nel modello base
17    peft_model = PeftModel.from_pretrained(base_model,
18        ADAPTER_PATH)
19    model = peft_model.merge_and_unload()
20    print("Fusione completata! Modello Datavault-LLM pronto...")
```

Code 4.5: Caricamento e fusione dell'adapter LoRA all'avvio del server FastAPI

Commentiamo due scelte architetturali degne di nota: `merge_and_unload()` fonde *fisicamente* i pesi dell'adapter LoRA nel modello base, anziché mantenerli separati durante l'inferenza, eliminando l'overhead computazionale di applicare l'aggiornamento di basso rango a ogni forward pass; e `device_map="cpu"` con `torch_dtype=torch.float32` rispecchiano un deployment senza GPU dedicate, dove la piena precisione a 32 bit evita instabilità numeriche nella fusione dei pesi (lo stesso tipo di instabilità osservato con precisioni ridotte durante il training).

La route di generazione

Ad ogni richiesta dell'utente, il backend espone l'endpoint `POST /generate`, che riceve la domanda in linguaggio naturale e restituisce la query SPARQL (o l'errore Out-of-Domain) generata dal modello:

```
1 @app.post("/generate")
2 async def generate_sparql(request: ChatRequest):
3     if not model or not tokenizer:
4         raise HTTPException(status_code=503, detail="Modello non
5         ancora caricato")
6     try:
7         # Costruzione dei messaggi basata sul System Prompt
8         # identico al training
9         messages = [
10             {"role": "system", "content": SYSTEM_PROMPT_TEXT},
11             {"role": "user", "content": request.message}
12         ]
13
14         # Uso del chat template nativo del modello
15         prompt = tokenizer.apply_chat_template(messages, tokenize=
16         False, add_generation_prompt=True)
17         inputs = tokenizer(prompt, return_tensors="pt")
18
19         model.eval()
20         end_token_id = tokenizer.convert_tokens_to_ids("<|end|>")
21         terminators = [tokenizer.eos_token_id, end_token_id]
22
23         # Inferenza pura senza gradienti
24         with torch.no_grad():
25             outputs = model.generate(
26                 **inputs,
27                 max_new_tokens=250,
28                 do_sample=False,
29                 pad_token_id=tokenizer.eos_token_id,
30                 eos_token_id=terminators
31             )
32
33         # Estrazione della risposta rimuovendo i token iniziali
34         input_length = inputs['input_ids'].shape[1]
35         generated_tokens = outputs[0][input_length:]
36         assistant_reply = tokenizer.decode(generated_tokens,
37         skip_special_tokens=True).strip()
38
39         # Segue il sanitizzatore
40         return {"response": sanitized_query}
```

Code 4.6: Route di generazione della query SPARQL

Abbiamo tre aspetti di questa implementazione particolarmente significativi:

- **Doppio token di stop:** il modello Phi-3 utilizza il token speciale `<|end|>` per segnalare la fine di un turno di conversazione, distinto dal token EOS standard del tokenizer. Includere *entrambi* in `terminators` si è rivelato necessario: in fase di sviluppo, l’omissione del token `<|end|>` causava generazione continuata oltre la query attesa, lo stesso fenomeno di hallucination continuata osservato con `packing=True` in training.
- **Generazione deterministica:** `do_sample=False` disabilita il campionamento stocastico, rendendo la generazione completamente deterministica. Per un compito di traduzione verso un linguaggio formale come SPARQL, dove esiste tipicamente un’unica forma corretta attesa, è preferibile il determinismo.
- **Limite di token generati:** `max_new_tokens=250` pone un tetto alla lunghezza della risposta generata, sufficiente per la forma di query vincolata dalla Regola 1 del system prompt (paragrafo 4.5.1), oltre a fungere da rete di sicurezza contro eventuali generazioni anomale che non si arrestino correttamente sui token di stop.

Il sanitizzatore: parsing e validazione dell’output

Il testo grezzo generato dal modello (`assistant_reply`) non viene restituito direttamente al frontend, ma passa attraverso una fase di post-processing che nel codice del backend è internamente chiamata “sanitizzatore”. Le responsabilità di questo passaggio includono:

- **Pulizia dei prefissi:** rimozione di eventuali blocchi `PREFIX` generati (o allucinati) dal modello, sostituiti dal blocco di prefissi hardcoded e correttamente allineati al namespace reale del Datavault.
- **Fallback Out-of-Domain:** riconoscimento e gestione del caso in cui il modello restituisca il JSON `{“error”: “OUT_OF_DOMAIN”}`, da intercettare prima di tentare un’esecuzione SPARQL che fallirebbe.
- **Fix del formato data:** normalizzazione del formato e del cast `STR()` nei filtri sulle date, per garantire coerenza con la Regola 6 del system prompt anche in caso di lievi variazioni nell’output del modello.

```
import re
import json

# Prefissi hardcoded
HARDCODED_PREFIXES = """
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX ldp: <http://www.w3.org/ns/ldp#>
PREFIX dc: <http://purl.org/dc/elements/1.1/>
PREFIX fedora: <http://fedora.info/definitions/v4/repository
#>
PREFIX ebucore: <http://www.ebu.ch/metadata/ontologies/
ebucore/ebucore#>
PREFIX exif: <http://www3org/2003/12/exif/ns#>
"""

def sanitize_sparql_query(raw_output: str) -> str:
    # 1. Gestione rapida delle richieste Fuori Dominio
    if "OUT_OF_DOMAIN" in raw_output:
        return '{"error": "OUT_OF_DOMAIN"}'
    # 2. Isolamento del corpo della query
    # Cerca la clausola SELECT e cattura tutto il testo
    match_select = re.search(r"(SELECT\s+DISTINCT.*)",
        raw_output,
        flags=re.IGNORECASE | re.DOTALL)

    if not match_select:
        # Fallback di sicurezza: se non trova la SELECT, ritorna l
        'output grezzo
        return raw_output

    core_query = match_select.group(1).strip()
    # 3. Fix difensivo sul formato date (Regola 6)
    # Assicura che la variabile della data sia castata a stringa
    dentro STRSTARTS
    core_query = re.sub(
        r'STRSTARTS\s*\((\s*\(?( [a-zA-Z0-9_ ]+)\s*,',
        r'STRSTARTS(STR(?\1),',
        core_query
    )
    # 4. Assemblaggio finale
    sanitized_query = f"{HARDCODED_PREFIXES}\n{core_query}"

    return sanitized_query
```

28

L'algoritmo di sanitizzazione opera come un filtro di sicurezza a cascata, applicando tre livelli di validazione:

Short-circuit evaluation (Out-of-Domain): Il primo controllo intercetta la presenza della stringa `OUT_OF_DOMAIN`.

Se individuata, la funzione si arresta restituendo il JSON formattato.

Questa scelta evita che l'engine tenti di fare il parsing di un messaggio di errore trattandolo come se fosse una stringa SPARQL.

Troncamento tramite Regex (`re.search`): L'LLM, pur addestrato a restituire solo la clausola `SELECT`, in alcuni casi limite tendeva ad anticipare la risposta con blocchi `PREFIX` parzialmente corretti. L'espressione regolare cattura esclusivamente la stringa a partire dal pattern `SELECT DISTINCT`, scartando tutto il testo antecedente. Successivamente, il blocco `HARDCODED_PREFIXES` provvede alla reiniezione dei namespace corretti.

Normalizzazione semantica tramite Regex (`re.sub`): Nonostante la chiarezza della Regola 6 nel *system prompt*, in rare occasioni il modello generava filtri temporali omettendo la conversione esplicita `STR(?date)`. Dato che i dati in Blazegraph richiedono un confronto lessicale puro su stringhe non standard, l'omissione di questo cast produceva una query formalmente valida ma vuota. L'espressione regolare applica una correzione tipografica "al volo", incapsulando dinamicamente la variabile temporale nel costrutto `STR()` nel caso in cui fosse assente.

4.5 Formato dati: dataset__datavault.jsonl

Il formato dati alla base del fine-tuning struttura la conoscenza del dominio in formato JSONL: ogni riga è un oggetto JSON contenente una tripla di messaggi (`system`, `user`, `assistant`), pronta per l'addestramento supervisionato tramite `SFTTrainer`.

4.5.1 Le 8 regole di generazione SPARQL

Per istruire l'LLM sono state definite 8 regole di generazione SPARQL, sotto forma di system prompt. Una caratteristica progettuale importante, nonché garanzia di coerenza tra training e inferenza, è che questo system prompt è identico sia in `genera_dataset.py` (training) sia in `main.py` (inferenza), eliminando una possibile fonte di *distribution shift* tra le condizioni di addestramento e quelle di utilizzo reale del modello:

```
1 Sei un traduttore da linguaggio naturale a SPARQL per un Data
  Vault Fedora. REGOLE ASSOLUTE:
2 1. Restituisci SEMPRE e SOLO: SELECT DISTINCT ?image WHERE {
  ... }. Nessun altra variabile nella SELECT.
3 2. Autore scatto: usa exif:artist. Autore caricamento: usa
  fedora:createdBy.
4 3. Dispositivo: usa exif:make.
5 4. Formato file: usa ebucore:hasMimeType.
6 5. Ricerca testuale: cerca SOLO dentro ebucore:filename.
  Attenzione: se l'utente cerca parole come "data", "del", "
  vault", trattale come testo, NON come date.
7 6. Date: usa exif:dateTime. Usa SEMPRE STR() nel filtro, es:
  FILTER(STRSTARTS(STR(?date), "2023:05:10")).
8 7. Ricerca in cartelle/container: usa ?container ldp:contains+
  ?image e dc:title sul container.
9 8. FUORI DOMINIO: Se la richiesta e' estranea all'archivio,
  rispondi SOLO con il JSON: {"error": "OUT_OF_DOMAIN"}.
```

Code 4.8: System prompt utilizzato in training e inference (identico in entrambi i file)

Queste regole impongono restrizioni al modello su quali relazioni attraversare, come filtrare le date, e come gestire la paginazione e i limiti dei risultati.

Se le regole devono essere modificate, è necessario aggiornarle in entrambi i punti della codebase: `main.py` per un impatto immediato in inferenza, e `genera_dataset.py` nel caso si voglia generare un nuovo dataset e ri-addestrare il modello, al fine di evitare la reintroduzione di un disallineamento tra training e inferenza.

4.5.2 Gestione degli errori: OUT_OF_DOMAIN

Per garantire l'affidabilità è fondamentale che il sistema sia in grado di ammettere i propri limiti. Per questo motivo, nel dataset è stata implementata un'architettura di *error handling* JSON-based: quando l'utente pone una domanda irrilevante

rispetto allo schema del Datavault, il modello è addestrato (Regola 8, paragrafo 4.5.1) a non tentare di forzare una query SPARQL, ma a restituire un esplicito stato `OUT_OF_DOMAIN`. Questo fallback previene l'esecuzione di query malformate e migliora la trasparenza dell'interfaccia utente.

Per riassumere, la tabella seguente elenca il comportamento del sistema nei principali casi di errore gestiti:

Caso di errore	Comportamento del sistema e strategia di mitigazione
Predicato allucinato dal modello	Prefissi hardcoded re-iniettati dal sanitizzatore (paragrafo 4.4.1); la validazione a valle blocca le query contenenti predicati non riconosciuti.
Query SPARQL malformata	La validazione sintattica blocca l'esecuzione prima dell'invio al triplestore; un errore strutturato viene restituito al frontend.
Richiesta Out-of-Domain	Il modello risponde esclusivamente con il JSON di fallback <code>{'error': 'OUT_OF_DOMAIN'}</code> (Regola 8).
Generazione ininterrotta (assenza di token EOS)	Adozione di un doppio token di stop (<code>< end ></code> + EOS standard) abbinato a un limite di sicurezza hardware (<code>max_new_tokens=250</code>).
Mismatch del namespace EXIF	Allineamento sistematico al namespace reale presente nei dati di produzione, rinunciando alla correzione canonica a monte.

Tabella 4.4: Riepilogo dei principali casi di errore gestiti dal sistema e relative strategie di mitigazione adottate.

4.6 Conclusioni del capitolo

Le scelte architetturali descritte in questo capitolo: dallo stack tecnologico basato su Fedora e Blazegraph, alla pipeline di fine-tuning con Unsloth e LoRA, fino all'implementazione del backend FastAPI e al raffinamento del dataset per gestire correttamente prefissi e casi limite; dimostrano come la transizione da un proof-of-concept teorico a uno strumento funzionante richieda un'attenta ingegnerizzazione del dato e del software.

Nel capitolo 5 esamineremo come questa architettura si comporta concretamente in fase di test quantitativo e qualitativo, incluso l'impatto del bug del namespace EXIF sulle metriche di precision e recall.

Capitolo 5

Valutazione

5.1 Introduzione al capitolo

Nel capitolo 4 abbiamo descritto come il sistema sia stato costruito; andremo adesso a valutarne il comportamento. Distinguiamo due piani di analisi: l'**efficienza** (paragrafo 5.2), ovvero le prestazioni quantitative misurabili (tempo di generazione, tasso di esecuzione corretta, convergenza del training) e l'**efficacia** (paragrafo 5.3), ovvero il miglioramento qualitativo apportato dal sistema rispetto alle soluzioni precedenti (funzionalità, usabilità e flessibilità).

5.1.1 Metodologia generale

La valutazione quantitativa è stata condotta secondo una pipeline a due fasi, eseguite su macchine diverse per ragioni di disponibilità hardware:

1. **Fase 1 — Generazione delle predizioni**
(`generate_predictions.py`): per ciascuna domanda di un test set di 500 esempi, il modello fine-tuned genera la query SPARQL corrispondente, registrando anche il tempo di generazione per esempio.
2. **Fase 2 — Valutazione contro Blazegraph**
(`evaluate_against_blazegraph.py`): sia la query generata sia quella di riferimento vengono eseguite contro l'endpoint Blazegraph reale del Datavault, e i rispettivi insiemi di risultati vengono confrontati per calcolare precision, recall e F1.

Abbiamo dovuto dividere la pipeline in due fasi perché la fase di generazione richiede l'accelerazione GPU (non disponibile localmente), mentre la fase di valutazione richiede solo chiamate I/O verso l'endpoint Blazegraph e può essere eseguita comodamente su CPU.

Nota metodologica: confronto sull'insieme completo dei risultati

Una scelta metodologica rilevante riguarda il trattamento delle clausole `ORDER BY` e `LIMIT`. SPARQL non garantisce un ordine deterministico dei risultati quando è presente un `LIMIT` senza un corrispondente `ORDER BY`: eseguire due volte la stessa query può restituire campioni diversi delle righe disponibili, producendo falsi negativi anche quando la query generata è testualmente identica a quella di riferimento.

Per questo motivo, precision e recall vengono calcolate sull'insieme *completo* dei risultati, dopo aver rimosso `ORDER BY` e `LIMIT` da entrambe le query prima dell'esecuzione. L'obiettivo è isolare la correttezza della *logica di filtro* (la clausola `WHERE`) — cioè se il modello ha capito *quali* elementi cercare — da un controllo distinto e separato sulla clausola `LIMIT`, registrato nella colonna `limit_match` del dataset di valutazione, che verifica se il modello ha capito anche *quanti* elementi erano richiesti.

5.2 Efficienza: valutazione quantitativa

5.2.1 Struttura dei test

Per ragioni di disponibilità hardware e per isolare i tempi di inferenza, i parametri di esecuzione del test sono stati strutturati come segue:

- **Macchina (generazione):** GPU NVIDIA T4 (su Kaggle), modello Phi-3-mini fine-tuned con adapter LoRA fuso (`merge_and_unload`).
- **Macchina (valutazione):** CPU locale. Nessun modello caricato; esegue unicamente chiamate HTTP verso l'endpoint Blazegraph via proxy Vite.
- **Task:** Traduzione di una domanda in italiano in una query SPARQL, confrontata contro il riferimento generato dal sistema a template (paragrafo 4.3.1).
- **Dati (Test Set):** 500 esempi. Costituisce una reale partizione *held-out* (estratta al 5% con seed deterministico `3407`), priva di sovrapposizioni col training.

5.2.2 Distribuzione del test set per categoria

Il test set estratto copre 7 delle 14 categorie di query definite nel generatore (paragrafo 4.3.1). Come visibile in tabella 5.1 e figura 5.1, la distribuzione non è uniforme: la categoria `data_esatta` da sola costituisce quasi la metà del campione (233/500, 46.6%), mentre alcune categorie di nicchia (es. `query_not_anno`) non risultano rappresentate.

Questo sbilanciamento va tenuto presente nell'interpretazione dei risultati: le metriche aggregate risultano molto più informative se lette trasversalmente per categoria (paragrafo 5.3.1) piuttosto che come media generale schiacciata dalla classe maggioritaria.

Categoria	N. esempi
data_esatta	233
formato_file	66
fuori_dominio	57
dispositivo	40
container_cartella	30
ricerca_testuale	28
autore	26
data_intervallo	20
Totale	500

Tabella 5.1: Distribuzione del test set per categoria euristica.

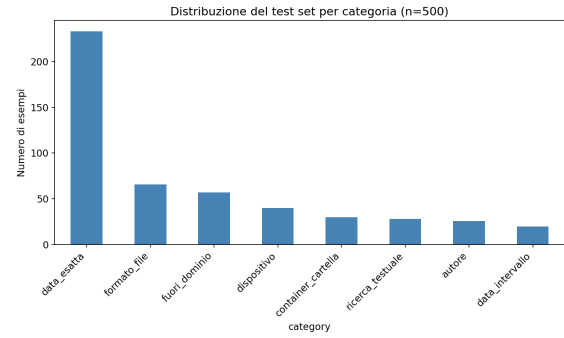


Figura 5.1: Distribuzione del test set di valutazione (n=500).

5.2.3 Tempo di generazione

Il tempo di generazione per query, misurato sulla GPU T4 durante la Fase 1, presenta le seguenti statistiche calcolate sull'intero set di 500 esempi:

Statistica	Valore (s)
Media	7.45
Deviazione standard	2.51
Minimo	0.65
25° percentile	7.64
Mediana	8.19
75° percentile	8.59
Massimo	10.45

Tabella 5.2: Statistiche del tempo di generazione per query (GPU T4).

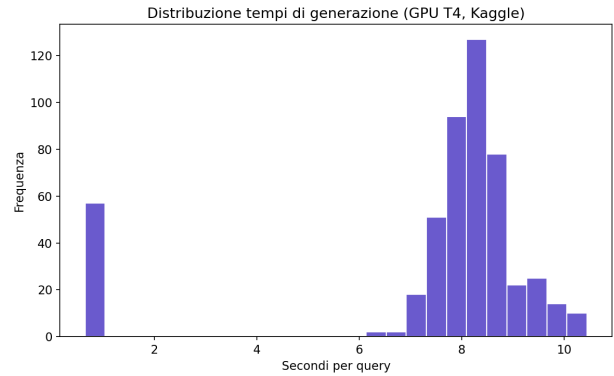


Figura 5.2: Distribuzione dei tempi di generazione su GPU T4.

La distribuzione è concentrata tra i 7 e i 9 secondi per la maggioranza delle query, con un'evidente coda di valori bassi (minimo 0.65s) verosimilmente corrispondenti a generazioni più brevi; come le risposte *Out-of-Domain*, strutturalmente più corte di una query SPARQL completa.

5.2.4 Tasso di esecuzione corretta

Su tutti i 443 esempi non Out-of-Domain del test set, il **100% delle query generate è stato eseguito con successo** contro l'endpoint Blazegraph reale, senza errori di sintassi o di esecuzione (`execution_ok = True` per tutte le righe, nessun `exec_error` registrato). Questo risultato conferma, a livello sintattico, l'efficacia delle 8 regole imposte tramite il system prompt (paragrafo 4.5.1) nel vincolare il modello a generare sempre query ben formate.

5.2.5 Convergenza del training

Il log di training (paragrafo 4.3.2) mostra una rapida convergenza della loss: da un valore di 0.044 allo step 50 fino a stabilizzarsi attorno a 0.010–0.011 a partire circa dallo step 400, mantenendosi sostanzialmente costante fino al termine dell’addestramento (step 1188, loss finale 0.010).

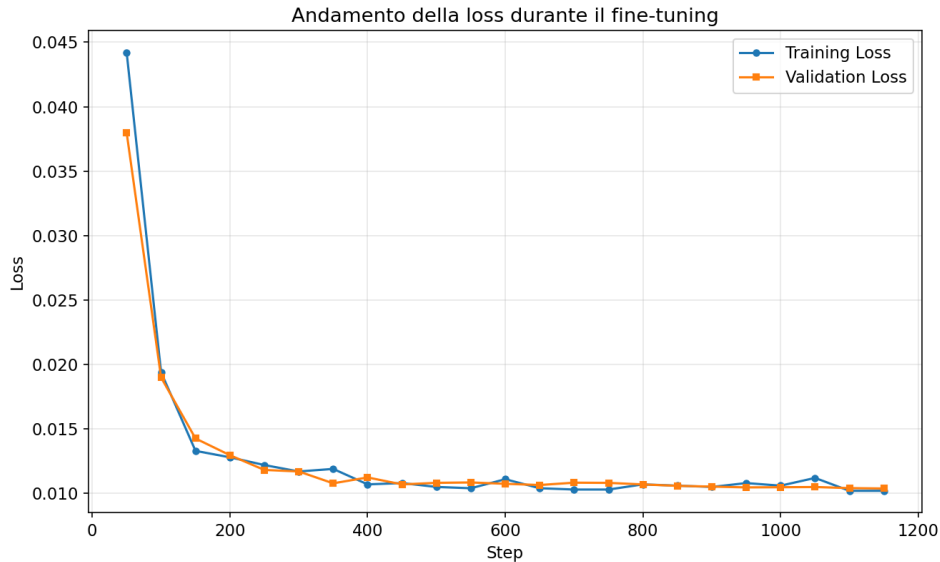


Figura 5.3: Andamento della training loss e della validation loss durante il fine-tuning

È rilevante osservare che la curva di *validation loss* segue da vicino quella di training senza una divergenza significativa (assenza di un evidente fenomeno di overfitting nella finestra di step osservata), e che entrambe le curve si stabilizzano ben prima del termine del training (intorno allo step 400 su un totale di 1188), suggerendo che il modello avrebbe potuto convergere anche con un numero inferiore di step.

5.3 Efficacia: valutazione qualitativa

5.3.1 Metriche di retrieval per categoria

Applicando la metodologia descritta in paragrafo 5.1.1 (confronto sull'insieme completo dei risultati, al netto di `ORDER BY / LIMIT`), le metriche di precision, recall e F1 per categoria sono le seguenti:

Categoria	N	Precision	Recall	F1
autore	26	1.000	1.000	1.000
container_cartella	30	1.000	1.000	1.000
data_esatta	233	1.000	1.000	1.000
data_intervallo	20	1.000	1.000	1.000
dispositivo	40	1.000	1.000	1.000
formato_file	66	1.000	1.000	1.000
ricerca_testuale	28	1.000	1.000	1.000
Media complessiva	443	1.000	1.000	1.000

Tabella 5.3: Precision, recall e F1 medi per categoria, calcolati sull'insieme completo dei risultati (al netto di `ORDER BY / LIMIT`)

Anche il riconoscimento delle richieste Out-of-Domain (paragrafo 4.5.2) raggiunge il 100% di accuratezza: tutti i 57 esempi fuori dominio nel test set sono stati correttamente classificati come tali dal modello. Allo stesso modo, il `limit_match` (corrispondenza esatta del valore richiesto nella clausola `LIMIT`) è verificato nel 100% dei 443 casi non Out-of-Domain in cui almeno una delle due query (generata o di riferimento) include un `LIMIT`.

5.3.2 Nota critica: interpretazione dei risultati perfetti

I risultati riportati in tabella 5.3 meritano una discussione esplicita, poiché un punteggio di 1.000 uniforme su ogni metrica e ogni categoria è un esito che richiede cautela interpretativa più che entusiasmo acritico.

Un'ispezione diretta delle 500 coppie (query generata, query di riferimento) in `predictions.json` rivela che **tutte le 500 query generate dal modello sono identiche, carattere per carattere, alla rispettiva query di riferimento**. Questo è un dato di fatto distinto, e più forte, della semplice equivalenza semantica dei risultati: non si tratta di formulazioni sintatticamente diverse che producono lo stesso insieme di risultati, ma di una riproduzione esatta della stringa SPARQL attesa.

Questo fenomeno è spiegabile alla luce di come il test set è stato costruito: le domande e le query di riferimento sono generate dallo stesso sistema di template parametrici (paragrafo 4.3.1) usato per produrre il dataset di training. Le 8 regole assolute del system prompt (paragrafo 4.5.1) vincolano fortemente la forma sintattica della query attesa—un'unica struttura `SELECT DISTINCT ?image WHERE {...}`, predicati fissi

per ciascun intento—riducendo lo spazio delle risposte plausibili a un insieme molto ristretto e, per costruzione, deterministico dato l'intento e i parametri della domanda.

In altre parole, una corrispondenza esatta al 100% è coerente con due ipotesi non mutuamente esclusive:

1. Il modello ha effettivamente appreso con grande efficacia la mappatura tra intenti linguistici e struttura SPARQL imposta dalle regole, generalizzando correttamente su parametri (anni, dispositivi, nomi di file) anche se non identici a quelli visti in training;
2. Il test set, essendo generato dallo stesso processo del training set, condivide con esso una distribuzione di superficie sufficientemente vincolata (per via delle 8 regole assolute) da rendere il compito di generazione quasi deterministico, riducendo il valore di questa valutazione come misura di *generalizzazione* a domande realmente nuove e formulate liberamente da un utente umano.

A seguito di una verifica sul codice di inferenza (`generate_predictions.ipynb`), è stato confermato che **il test set costituisce una reale partizione *held-out***. L'estrazione è avvenuta in modo deterministico applicando un rapporto 95%/5% e utilizzando il medesimo seed (`3407`) impiegato per la configurazione del training. Questa garanzia metodologica esclude la presenza di *data leakage* o sovrapposizioni: il modello ha elaborato istanze effettivamente mai viste in fase di addestramento. Ciò avvalorava l'ipotesi della reale generalizzazione strutturale (punto 1), pur confermando che tale generalizzazione avviene strettamente *in-distribution* rispetto ai template.

Questa osservazione non invalida il risultato—un'accuratezza sintattica ed esecutiva del 100% sui formati di query coperti dal generatore è comunque un traguardo solido e utile per l'applicativo—ma ne delimita correttamente la portata: la valutazione quantitativa qui presentata misura primariamente la capacità del modello di rispettare in modo affidabile e ripetibile la grammatica e le regole imposte dal system prompt, più che la sua capacità di decodificare formulazioni libere e impreviste di un ricercatore umano. Una valutazione complementare con domande poste in modo spontaneo da utenti reali, non vincolate dai template del generatore, costituisce un'estensione naturale di questo lavoro (capitolo 6).

5.3.3 Miglioramento rispetto alle soluzioni precedenti

Al di là delle metriche quantitative, l'efficacia del sistema si misura anche rispetto al miglioramento qualitativo delle funzionalità disponibili per il ricercatore umanista, in continuità con la discussione del capitolo 2 (architettura dell'informazione, berrypicking, carico cognitivo):

- **Funzionalità:** prima dell'introduzione del sistema, non esisteva alcun meccanismo di ricerca per contenuto o concetto sul Datavault; l'unica modalità di accesso era la navigazione diretta della struttura a container (paragrafo 3.2.1). Il sistema introduce ex novo questa funzionalità.

- **Usabilità:** l'interazione è ridotta a un singolo campo di testo in linguaggio naturale, eliminando la necessità di conoscere prefissi RDF, sintassi SPARQL o la struttura interna dello schema (paragrafo 2.2.3).
- **Integrazione:** il sistema si integra nell'infrastruttura esistente (Fedora/Blazegraph) senza richiedere modifiche allo schema dati o migrazioni, operando come layer applicativo aggiuntivo.
- **Manutenzione:** la separazione netta tra dataset di training, system prompt e backend di inferenza (capitolo 4) consente di aggiornare il comportamento del sistema (ad esempio aggiungendo nuove categorie di query) senza dover intervenire sull'infrastruttura dati sottostante.

5.4 Discussione dei risultati e limiti della valutazione

I risultati presentati in questo capitolo possono essere riassunti in tre punti principali:

1. Il sistema è **sintatticamente affidabile**: il 100% delle query generate nel test set è eseguibile senza errori contro Blazegraph, grazie al vincolo imposto dalle 8 regole del system prompt.
2. Il sistema è **efficiente nei tempi di risposta**: una media di 7.45 secondi per query su GPU T4 è compatibile con un'interazione conversazionale, sebbene il confronto con un'esecuzione su CPU richieda ulteriore verifica empirica (paragrafo 5.2.3).
3. I punteggi di precision/recall/F1 pari a 1.000 vanno interpretati con cautela metodologica (paragrafo 5.3.2): riflettono primariamente la capacità del modello di rispettare un formato di query fortemente vincolato e generato dalla stessa famiglia di template del training set, più che una misura di generalizzazione su domande realmente libere.

Il limite principale di questa valutazione è dunque la natura **sintetica e in-dominio** del test set utilizzato. Una valutazione più conclusiva richiederebbe un confronto con domande poste da utenti reali del dominio DH.ARC, non vincolate dalla struttura dei template di generazione—un'estensione discussa come sviluppo futuro nel capitolo 6.

5.5 Conclusioni del capitolo

In questo capitolo abbiamo valutato il sistema lungo due assi complementari. Sul piano dell'**efficienza**, il sistema mostra tempi di generazione contenuti (media 7.45s su GPU), un tasso di esecuzione corretta del 100% e una rapida convergenza in fase di training (loss stabilizzata già a circa un terzo del training totale). Sul piano dell'**efficacia**, le metriche di retrieval (precision, recall, F1) raggiungono il valore massimo su tutte le categorie testate, un risultato che—come discusso criticamente in paragrafo 5.3.2—va attribuito principalmente all'efficacia delle regole di vincolo sintattico imposte dal system prompt e alla natura in-distribuzione del test set, più che a una dimostrata capacità di generalizzazione su input completamente liberi.

Il capitolo 6 riprende questi risultati per bilanciare i punti di forza del sistema con i suoi limiti noti, e per delineare gli sviluppi futuri necessari a una valutazione più rigorosa e a un utilizzo in produzione del sistema.

Capitolo 6

Conclusioni e sviluppi futuri

6.1 Risultati raggiunti

6.2 Motivazioni di orgoglio e modestia

6.2.1 Punti di forza e orgoglio

6.2.2 Limiti del sistema e punti di modestia

6.3 Sviluppi futuri

6.3.1 Bug fix e ottimizzazioni a breve termine

6.3.2 Estensioni fondamentali e feature mancanti

6.3.3 Visioni di lungo termine (Desideri e sogni)

Riferimenti bibliografici

- [ALM26] ALMA MATER STUDIORUM - Università di Bologna. /*DH.ARC - Digital Humanities Advanced Research Centre*. <https://centri.unibo.it/dharc/en>, 2026.
- [Bat89] Marcia J. Bates. The design of browsing and berrypicking techniques for the online search interface. *Online Review*, 13(5):407–424, 1989.
- [FD25] Mohammad Javad Farokhi Darani. Metadata management with semantic etls: From production repositories to dissemination environments. Master’s thesis, Alma Mater Studiorum - Università di Bologna, 2025.
- [Mar25] Mohammed Maree. Quantifying relational exploration in cultural heritage knowledge graphs with llms: A neuro-symbolic approach for enhanced knowledge discovery. *Data*, 10(4):52, 2025.
- [MR08] Peter Morville and Louis Rosenfeld. *Information Architecture for the World Wide Web*. O’Reilly Media, Sebastopol, CA, 3 edition, 2008.
- [PT22] Silvio Peroni and Francesca Tomasi. Approaching digital humanities at university: a cultural challenge. *Journal of Art Historiography*, 27, December 2022.
- [Ros07] Luca Rosati. *Architettura dell’informazione: Guida alla trovabilità, dagli oggetti quotidiani al Web*. Apogeo, 2007.
- [Tol70] J. R. R. Tolkien. *Il Signore degli Anelli*. Bompiani, 1970.
- [Vaš24] Linas Vaštakas. Cultural heritage search with large language models: Enhancing the discoverability of cultural heritage artifacts through large language model-based search systems. Master’s thesis, Linnaeus University, Sweden, 2024.
- [VFQP25] Iva Vasic, Hans-Georg Fill, Ramona Quattrini, and Roberto Pierdicca. Knowledge graphs vs. large language models: Competitors or partners in supporting virtual museums. *ACM Journal on Computing and Cultural Heritage*, 18(4), dec 2025.

Ringraziamenti

Grazie a tutt*.